Министерство образования и науки Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет «Программной инженерии и компьютерной техники»

Проектирование вычислительных систем

Лабораторная работа №1

Выполнил:
Григорьев Давид Владимирович, Тынкевич Валерий Николаевич
Группа: P3415

Проверил:
Ключев Владислав Аркадьевич

Санкт-Петербург, 2026

1 Задание

Вариант: $12 \bmod 5 = 2$ — «Гирлянда».

Текст задания приведён из учебного пособия А. О. Ключева, В. Ю. Пинкевича, А. Е. Платунова, В. А. Ключева «Стенд-конструктор SDK-1.1M. Организация и программирование микроконтроллеров», редакция от 23.09.2026, раздел 5.1, с. 74–75, 77.

1.1 Часть 1. Разработка прикладной программы для стенда с использованием GPIO.

Разработать и реализовать драйверы управления светодиодными индикаторами и чтения состояния кнопки стенда SDK-1.1M, расположенных на боковой панели стенда. Обработка нажатия кнопки в программе должна включать программную защиту от дребезга. Функции и другие компоненты драйверов должны быть универсальными, т. е. пригодными для использования в любом из вариантов задания и не должны содержать прикладной логики программы. Функции драйверов должны быть неблокирующими, то есть не должны содержать ожиданий события (например, нажатия кнопки). Также, в драйверах не должно быть пауз с активным ожиданием (функция HAL_Delay() и собственные варианты с аналогичной функциональностью).

При необходимости параллельного выполнения разных процессов (например, опроса кнопки и ожидания перед переключением светодиодов) следует организовать псевдопараллельную работу процессов в стиле кооперативной многозадачности. Это возможно сделать без существенной потери точности соблюдения необходимых интервалов времени между действиями, поскольку действия выполняются очень быстро по сравнению с промежутками ожидания между ними. Каждый процесс (который может быть выражен всего в нескольких строках кода) должен использовать не блокирующее ожидание (функция HAL_Delay()), а считывать текущее значение миллисекундного счетчика (функция HAL_GetTick()), проверяя, прошло ли необходимое количество времени для выполнения следующего действия. После проверки и (при необходимости) выполнения действия управление должно передаваться следующему процессу, чтобы он тоже мог провести такую проверку. Функция HAL_Delay() может применяться в отдельных случаях, когда блокирующее ожидание не мешает реализации остальной логики программы.

Контакты подключения кнопки и светодиодов должны быть настроены в режиме GPIO. Использование прерываний и дополнительных таймеров (кроме SysTick) не допускается.

1.2 Часть 2. Изучение логики работы портов ввода-вывода GPIO в STM32.

Данная часть выполняется одним из двух способов на выбор исполнителей.

1.2.1 Способ 1.

Разработать собственную реализации драйвера GPIO (не HAL и LL) и внедрить ее в разработанную ранее программу для стенда.

Требования к драйверу:

- должны быть реализованы функции инициализации режимов работы портов ввода-вывода, получения текущих настроек режима работы порта, чтения и записи значений как единичных сигналов порта, так и одновременное нескольких сигналов в пределах одного порта;
- должна поддерживаться повторная инициализация портов с изменением режима работы, при этом не должно допускаться кратковременных «выбросов» сигналов (переключений в некорректное состояние) на выходных портах в процессе записи значений в управляющие регистры;
- поддержка прерываний не требуется;
- допускается использование определенной в HAL структуры регистров типа GPIO_TypeDef и define-определений; остальные структуры и функции для работы с GPIO использовать нельзя, требуется разработать свои варианты.

При внедрении разработанного драйвера необходимо удалить из программы все обращения к функциям стандартных библиотек HAL и LL для настройки и использования GPIO, в том числе необходимо отключить инициализацию ножек GPIO, используемых для управления светодиодами и кнопкой, в графическом конфигураторе (файл *.ioc). Все должно выполняться с помощью функций разработанного драйвера.

Для защиты требуется уметь детально объяснить, чем разработанный драйвер отличается от стандартных драйверов HAL и LL.

1.2.2 Способ 2.

Разработать программную симуляцию работы части аппаратуры микроконтроллера STM32F427VGT6, отвечающей за управление портами ввода-вывода GPIO – управляющие регистры GPIO и логика, обеспечивающая их сопряжение с физическими портами ввода-вывода.

Требования:

- состав регистров, назначение их полей и логика должны соответствовать справочному руководству (RM0090 Reference manual [4], раздел «General purpose I/O (GPIO)»);
- поддержка прерываний и симуляция настройки синхросигналов не требуется – считать, что готовый синхросигнал подается извне симулируемого блока;
- симулятор должен поддерживать операции типа «записать в регистр» и «прочитать регистр» – например, в виде вызовов функций;
- симулятор должен поддерживать программируемое изменение состояния для входных портов в заданные моменты времени;
- с помощью данных возможностей должна задаваться последовательность операций чтения и записи регистров, имитирующая работу программы микроконтроллера.
- после завершения (при возможности также в процессе) работы симулятора должна быть обеспечена возможность проверить корректность его работы с помощью сохраненных текстовых журналов и/или графического интерфейса, в которых видно выполнение логики задания.

В рамках симуляции описать последовательность, которая имитирует логику, близкую к заданию лабораторной работы по варианту.

Выбор технологий для реализации – на усмотрение исполнителей. Возможные варианты:

- языки моделирования аппаратного и программного обеспечения (например, SystemC);
- использование готового симулятора процессорного ядра типа QEMU с собственными расширениями для моделирования периферии;
- собственные решения на базе языков программирования общего назначения, в том числе веб-технологий.

1.3 Вариант 2. «Гирлянда»

Реализовать простой имитатор гирлянды с переключением режимов. Должно быть реализовано не менее четырех последовательностей переключения светодиодов, обязательно с разной частотой мигания (должны быть визуально отличимыми) и разными последовательностями цветов. В каждом режиме задействуется не менее двух светодиодов, которые могут зажигаться одновременно или по отдельности. При каждом нажатии кнопки должно происходить переключение режима на следующий. По достижении последнего режима должен происходить возврат к первому режиму. При повторном входе в какой-либо режим анимация на светодиодах должна запускаться с того места, на котором в прошлый раз была прервана переключением на следующий режим. Удерживание зажатой кнопки не должно приводить к «повисанию» анимации или непрерывному переключению режимов. На одно нажатие кнопки должно происходить одно переключение.

2 Исходный код

2.1 Программа микроконтроллера

Файл `main.c` выбирает вариант программы и передаёт управление соответствующей реализации. В данной сборке выбран вариант 1.

```
      #include "board.h"

static const int assignment_part = 1;
volatile uint32_t system_millis;

void SysTick_Handler(void)
{
    ++system_millis;
}

static void gpio_init(void)
{
    *RCC_AHB1ENR |= (1U << 2U) | (1U << 3U);
    (void)*RCC_AHB1ENR;

    *GPIOD_BSRR = (uint32_t)LED_MASK << 16U;
    for (uint32_t pin = GREEN_PIN; pin <= RED_PIN; ++pin) {
        *GPIOD_MODER &= ~(3U << (pin * 2U));
        *GPIOD_OTYPER &= ~(1U << pin);
        *GPIOD_OSPEEDR &= ~(3U << (pin * 2U));
```

```
        *GPIOD_PUPDR &= ~(3U << (pin * 2U));
        *GPIOD_MODER |= (1U << (pin * 2U));
    }

    *GPIOC_MODER &= ~(3U << (BUTTON_PIN * 2U));
    *GPIOC_PUPDR &= ~(3U << (BUTTON_PIN * 2U));
    *GPIOC_PUPDR |= (1U << (BUTTON_PIN * 2U));
}

static void systick_init(void)
{
    *SYST_RVR = (CORE_CLOCK_HZ / 1000U) - 1U;
    *SYST_CVR = 0U;
    *SYST_CSR = 7U;
}

int main(void)
{
    gpio_init();
    systick_init();

    if (assignment_part == 1) {
        part1_run();
    } else {
        part2_run();
    }

    for (;;) {
    }
}
```

В таблице режимов заданы состояния светодиодов и периоды переключения. Индекс и время последнего изменения хранятся отдельно для каждого режима, поэтому при возвращении к нему последовательность продолжается с прежнего места. Функция обработки кнопки фильтрует дребезг программно, а главный цикл проверяет миллисекундный счетчик без блокирующих задержек.

```
    #include "board.h"

typedef struct {
    uint32_t states[4];
    int length;
    int index;
    uint32_t period_ms;
    uint32_t changed_at;
```

```
} GarlandMode;

static int active_mode;
static bool button_candidate;
static bool button_state;
static uint32_t button_changed_at;

static GarlandMode modes[4] = {{{GREEN, RED, YELLOW, GREEN}, 4, 0, 200, 0},
                               {{RED, YELLOW, GREEN, RED}, 4, 0, 500, 0},
                               {{GREEN | YELLOW, RED, GREEN | RED, YELLOW}, 4,
0, 350, 0},
                               {{RED | YELLOW, GREEN, GREEN | YELLOW, RED}, 4,
0, 800, 0}};

static void show_state(uint32_t state)
{
    *GPIOD_BSRR = ((LED_MASK & ~state) << 16) | state;
}

static void select_next_mode(uint32_t now)
{
    active_mode = (active_mode + 1) % 4;
    modes[active_mode].changed_at = now;
    show_state(modes[active_mode].states[modes[active_mode].index]);
}

void part1_run(void)
{
    show_state(modes[0].states[0]);

    for (;;) {
        const uint32_t now = system_millis;
        const bool pressed = (*GPIOC_IDR & (1U << BUTTON_PIN)) == 0;

        if (pressed != button_candidate) {
            button_candidate = pressed;
            button_changed_at = now;
           } else if (button_candidate != button_state && (uint32_t)(now -
button_changed_at) >= 30) {
            button_state = button_candidate;
            if (button_state) {
                select_next_mode(now);
            }
        }
```

```
        GarlandMode *mode = &modes[active_mode];
        if ((uint32_t)(now - mode->changed_at) >= mode->period_ms) {
            mode->changed_at += mode->period_ms;
            mode->index = (mode->index + 1) % mode->length;
            show_state(mode->states[mode->index]);
        }
    }
}
```

2.2 Симулятор GPIO

Симулятор воспроизводит регистры GPIO STM32F427 и физические уровни контактов. Демонстрационный сценарий настраивает PC15 как вход кнопки с подтяжкой, а PD13–PD15 как выходы светодиодов; затем задаёт нажатия кнопки во времени и переключает режимы гирлянды.

2.2.1 Сценарий гирлянды

```
      """Демонстрация GPIO: вариант 2 лабораторной — гирлянда и кнопка PC15."""
from pathlib import Path
from stm32_gpio import BSRR, IDR, MODER, OTYPER, OSPEEDR, PUPDR, Simulator

LED_PINS = (13, 14, 15)  # green, yellow, red (плата SDK-1.1M)
LED_MASK = sum(1 << pin for pin in LED_PINS)
BUTTON = 15
MODES = (("g", "r", "y", "n"), ("g", "y", "r", "n"), ("r", "n", "y", "n", "g",
"n"), ("g", "g", "n", "y", "n", "r", "n"))
PERIODS = (200, 500, 1000, 350)
STATE_PIN = {"g": 13, "y": 14, "r": 15}


def configure(sim: Simulator) -> None:
    c, d = sim.port("C"), sim.port("D")
    # PC15: input + pull-up, therefore unpressed button reads 1.
    c.write(MODER, c.read(MODER) & ~(3 << 30))
    c.write(PUPDR, c.read(PUPDR) | (1 << 30))
    # PD13..15: push-pull outputs, low speed, without pull resistors.
    for pin in LED_PINS:
        d.write(MODER, (d.read(MODER) & ~(3 << (2 * pin))) | (1 << (2 * pin)))
        d.write(OTYPER, d.read(OTYPER) & ~(1 << pin))
        d.write(OSPEEDR, d.read(OSPEEDR) & ~(3 << (2 * pin)))
        d.write(PUPDR, d.read(PUPDR) & ~(3 << (2 * pin)))
    d.write(BSRR, LED_MASK << 16)


def show(sim: Simulator, state: str) -> None:
```

```
    d = sim.port("D")
    d.write(BSRR, LED_MASK << 16)  # atomic reset of all LEDs
     if state != "n": d.write(BSRR, 1 << STATE_PIN[state])  # atomic set of
one LED
         sim.lines.append(f"{sim.time_ms:06d}  ms  APP:  state={state};  LEDs
physical=0x{d.physical_levels() & LED_MASK:04X}")


def run() -> Simulator:
    sim = Simulator(); configure(sim)
   # Presses are active-low, scheduled independently from application polling.
      sim.schedule_input(750, "C", BUTTON, 0); sim.schedule_input(810, "C",
BUTTON, 1)
     sim.schedule_input(1700, "C", BUTTON, 0); sim.schedule_input(1760, "C",
BUTTON, 1)
    mode = index = 0; changed_at = 0; previous_pressed = False
    show(sim, MODES[mode][index])
    for now in range(0, 2601, 10):
        sim.advance_to(now)
        pressed = not bool(sim.port("C").read(IDR) & (1 << BUTTON))
        if pressed and not previous_pressed:
            mode = (mode + 1) % len(MODES); index = 0; changed_at = now
             sim.lines.append(f"{now:06d} ms APP: PC15 pressed -> mode {mode
+ 1}")
            show(sim, MODES[mode][index])
        previous_pressed = pressed
        if now - changed_at >= PERIODS[mode]:
           changed_at += PERIODS[mode]; index = (index + 1) % len(MODES[mode])
            show(sim, MODES[mode][index])
    return sim


if __name__ == "__main__":
    simulator = run()
    destination = Path(__file__).with_name("garland.log")
    simulator.save_log(destination)
    print(f"Simulation completed: {destination}")
                print("Final    GPIOC.IDR=0x%04X,    GPIOD.ODR=0x%04X"    %
(simulator.port("C").read(IDR), simulator.port("D").read(0x14)))
```

2.2.2 Модель регистров и внешних входов

```
     """Регистровая модель GPIO STM32F427 (RM0090, раздел 8.4).

Модель намеренно не содержит EXTI и RCC: тактирование считается поданным
снаружи, как и указано в задании. Время измеряется в миллисекундах и задаётся
```

```
классом Simulator.
"""
from __future__ import annotations

from dataclasses import dataclass, field
from pathlib import Path
from typing import Callable, Optional

PIN_MASK = 0xFFFF

MODER, OTYPER, OSPEEDR, PUPDR, IDR, ODR, BSRR, LCKR, AFRL, AFRH = (
    0x00, 0x04, 0x08, 0x0C, 0x10, 0x14, 0x18, 0x1C, 0x20, 0x24
)
REGISTER_NAMES = {
    MODER: "MODER", OTYPER: "OTYPER", OSPEEDR: "OSPEEDR", PUPDR: "PUPDR",
     IDR: "IDR", ODR: "ODR", BSRR: "BSRR", LCKR: "LCKR", AFRL: "AFRL", AFRH:
"AFRH",
}


class GPIOError(ValueError):
    """Неверный адрес или недопустимая операция с GPIO."""


@dataclass
class GPIOPort:
    """Один 16-разрядный порт GPIOx.

    Внешний уровень ``None`` означает высокоимпедансную внешнюю цепь. Он
    разрешается внутренней подтяжкой. Режимы MODER: 00 input, 01 output,
    10 alternate function, 11 analog.
    """

    name: str
    now: Callable[[], int]
    log: Callable[[str], None]
    moder: int = 0
    otyper: int = 0
    ospeedr: int = 0
    pupdr: int = 0
    odr: int = 0
    lckr: int = 0
    afrl: int = 0
    afrh: int = 0
    _lock_phase: int = 0
```

```
    external: list[Optional[int]] = field(default_factory=lambda: [None] * 16)
    alternate_output: int = 0

    def _emit(self, message: str) -> None:
        self.log(f"{self.now():06d} ms {self.name}: {message}")

    @staticmethod
    def _field(value: int, pin: int) -> int:
        return (value >> (2 * pin)) & 0b11

    def _locked_mask(self) -> int:
        return self.lckr & PIN_MASK if (self.lckr & (1 << 16)) else 0

    def _write_2bit(self, old: int, value: int) -> int:
        """Не меняет поля конфигурации закреплённых LCKR выводов."""
        locked = self._locked_mask()
        result = old
        for pin in range(16):
            if not (locked & (1 << pin)):
                  result = (result & ~(3 << (2 * pin))) | (value & (3 << (2
* pin)))
        return result & 0xFFFFFFFF

    def _write_af(self, old: int, value: int, first_pin: int) -> int:
        """AFRL/AFRH: по 4 бита AFy на контакт, с учётом LCKR."""
        result = old
        locked = self._locked_mask()
        for local_pin in range(8):
            if not (locked & (1 << (first_pin + local_pin))):
                shift = 4 * local_pin
                result = (result & ~(0xF << shift)) | (value & (0xF << shift))
        return result & 0xFFFFFFFF

    def _line_level(self, pin: int) -> int:
        mode = self._field(self.moder, pin)
        ext = self.external[pin]
        # Output driver is connected in output and alternate-function modes.
        if mode in (1, 2):
            driven = (self.odr if mode == 1 else self.alternate_output) >> pin
& 1
            open_drain = (self.otyper >> pin) & 1
            if not open_drain or driven == 0:
                return driven
          # Open drain releases a logical '1'; the line is determined outside.
        if ext is not None:
```

```
            return ext
        pull = self._field(self.pupdr, pin)
          return 1 if pull == 1 else 0  # pull-up; pull-down/floating -> 0
in model

    def _idr(self) -> int:
        return sum(self._line_level(pin) << pin for pin in range(16))

    def read(self, offset: int) -> int:
        if offset == MODER: value = self.moder
        elif offset == OTYPER: value = self.otyper
        elif offset == OSPEEDR: value = self.ospeedr
        elif offset == PUPDR: value = self.pupdr
        elif offset == IDR: value = self._idr()
        elif offset == ODR: value = self.odr
        elif offset == BSRR: value = 0  # write-only
        elif offset == LCKR:
            # First and second reads complete the LCKK write/read sequence.
            if self._lock_phase == 3:
                self._lock_phase = 4
            elif self._lock_phase == 4:
                self.lckr |= 1 << 16
                self._lock_phase = 5
            value = self.lckr
        elif offset == AFRL: value = self.afrl
        elif offset == AFRH: value = self.afrh
      else: raise GPIOError(f"{self.name}: неизвестное смещение 0x{offset:X}")
        self._emit(f"READ  {REGISTER_NAMES[offset]:5s} -> 0x{value:08X}")
        return value

    def write(self, offset: int, value: int) -> None:
        value &= 0xFFFFFFFF
        if offset == MODER: self.moder = self._write_2bit(self.moder, value)
        elif offset == OTYPER:
             lock = self._locked_mask(); self.otyper = (self.otyper & lock) |
(value & ~lock & PIN_MASK)
        elif offset == OSPEEDR: self.ospeedr = self._write_2bit(self.ospeedr,
value)
       elif offset == PUPDR: self.pupdr = self._write_2bit(self.pupdr, value)
        elif offset == ODR:
            lock = self._locked_mask(); self.odr = (self.odr & lock) | (value
& ~lock & PIN_MASK)
        elif offset == BSRR:
            # BSy has priority if set and reset are written for same pin.
            set_bits, reset_bits = value & PIN_MASK, (value >> 16) & PIN_MASK
```

```
            self.odr = ((self.odr & ~reset_bits) | set_bits) & PIN_MASK
        elif offset == LCKR:
            # RM0090: LCKK=1, LCKK=0, LCKK=1, read, read. Until then no lock.
            if self._lock_phase == 5:
              self._emit(f"WRITE LCKR  ignored (locked); ODR=0x{self.odr:04X};
IDR=0x{self._idr():04X}")
                return
            candidates = value & PIN_MASK
            lckk = bool(value & (1 << 16))
            if self._lock_phase == 0 and lckk:
                self.lckr = candidates; self._lock_phase = 1
            elif self._lock_phase == 1 and not lckk:
                self._lock_phase = 2
             elif self._lock_phase == 2 and lckk and candidates == (self.lckr
& PIN_MASK):
                self._lock_phase = 3
          # A new complete sequence is the only way to change the selected pins.
        elif offset == AFRL: self.afrl = self._write_af(self.afrl, value, 0)
        elif offset == AFRH: self.afrh = self._write_af(self.afrh, value, 8)
                  elif  offset  in  (IDR,):  raise  GPIOError(f"{self.name}:
{REGISTER_NAMES[offset]} доступен только для чтения")
      else: raise GPIOError(f"{self.name}: неизвестное смещение 0x{offset:X}")
            self._emit(f"WRITE  {REGISTER_NAMES[offset]:5s}  <-  0x{value:08X};
ODR=0x{self.odr:04X}; IDR=0x{self._idr():04X}")

    def set_input(self, pin: int, level: Optional[int]) -> None:
        if not 0 <= pin < 16 or level not in (None, 0, 1):
            raise GPIOError("pin должен быть 0..15, level — 0, 1 или None")
        self.external[pin] = level
        shown = "Z" if level is None else str(level)
            self._emit(f"PIN     P{self.name[-1]}{pin}  external  <-  {shown};
IDR=0x{self._idr():04X}")

    def physical_levels(self) -> int:
        """Фактические уровни на 16 физических контактах."""
        return self._idr()


class Simulator:
    """Диспетчер модельного времени, портов, внешних событий и журнала."""
    def __init__(self) -> None:
        self.time_ms = 0
        self.lines: list[str] = []
                self.ports  =  {name:  GPIOPort(name,  lambda:  self.time_ms,
self.lines.append)
```

```
                    for name in "ABCDEFGHIJK"}
        self._events: list[tuple[int, str, int, Optional[int]]] = []

    def port(self, name: str) -> GPIOPort:
        try: return self.ports[name.upper()]
           except KeyError as exc: raise GPIOError(f"Нет порта GPIO{name}")
from exc

       def schedule_input(self, at_ms: int, port: str, pin: int, level:
Optional[int]) -> None:
         if at_ms < self.time_ms: raise GPIOError("Нельзя назначить событие
в прошлом")
        self._events.append((at_ms, port.upper(), pin, level))
        self._events.sort(key=lambda event: event[0])

    def advance_to(self, target_ms: int) -> None:
      if target_ms < self.time_ms: raise GPIOError("Время не может идти назад")
        while self._events and self._events[0][0] <= target_ms:
            at, port, pin, level = self._events.pop(0)
            self.time_ms = at
            self.port(port).set_input(pin, level)
        self.time_ms = target_ms

    def save_log(self, filename: str | Path) -> None:
                  Path(filename).write_text("\n".join(self.lines)  +  "\n",
encoding="utf-8")
```